IFCO

Data Engineering Challenge

Results Report

Candidate: Sergio Moya Copa
Stack: Databricks Community Edition · PySpark · Power BI
June 2026

Executive Summary

All five mandatory tests have been completed and produce correct results on the provided dataset. Business logic is separated into pure Python functions in `src/transformations.py` to enable unit testing without a Spark cluster. 27 unit tests pass across the five test files. A Power BI dashboard (Test 6) covers the three requested business questions.

5 / 5	27 / 27	Done	4
Tests completed	Unit tests passing	Test 6 Dashboard	Data issues handled

Data Quality Findings (EDA)

A preliminary exploratory analysis was conducted before solving any exercise. The following issues were identified and addressed in the solution:

#	Finding	Impact	Resolution
1	contact_data in 3 formats (empty, array [{...}], object {...})	Tests 2 & 3	parse_contact_json() handles all 3 formats
2	Duplicate invoice for order ...d487 (same amount)	Test 4	ROW_NUMBER() dedup before join
3	Dirty company name variants ('Fresh Fruits Co' / 'Fresh Fruits c.o')	Tests 1 & 5	normalize_company_name() lowercase + strip non-alphanum
4	Amounts in cents, VAT as string (grossValue=345310, vat='34')	Test 4	Cast + net = gross / (1 + vat/100) / 100

Notebook Results

Test 1 — Crate Type Distribution per Company

Number of orders per crate type for each company, with company name normalisation to consolidate dirty duplicates. A percentage breakdown per company is also included.

company_name_clean	crate_type	num_orders	pct_crate_by_company
acme corp	Metal	2	40.0%
acme corp	Plastic	2	40.0%
acme corp	Wood	1	20.0%
fresh fruits co	Plastic	3	75.0%
fresh fruits co	Wood	1	25.0%
global greens	Metal	1	33.3%
global greens	Plastic	1	33.3%
global greens	Wood	1	33.3%
...	...	...	...

Test 2 — Contact Full Name (df_1)

Extracts contact_name and contact_surname from the embedded JSON. Returns 'John Doe' when data is missing, malformed or empty strings.

order_id (truncated)	contact_full_name
...2a3b4c	Curtis Jackson
...5d6e7f	Ana Lopez
...8g9h0i	John Doe ← placeholder (null)
...1j2k3l	John Doe ← placeholder (malformed JSON)

Validation: 0 nulls in contact_full_name column. Placeholder rate consistent with null count found in EDA.

Test 3 — Contact Address (df_2)

Extracts city and postal code in format 'city, cp'. cp is cast to string to handle both integer and string values in source data.

order_id (truncated)	contact_address
...2a3b4c	Munich, 80331
...5d6e7f	Berlin, 10115
...8g9h0i	Unknown, UNK00 ← both placeholders
...1j2k3l	Madrid, UNK00 ← cp missing
...4m5n6o	Unknown, 28001 ← city missing

Test 4 — Sales Team Commissions

Commissions are calculated on net invoice value (gross / (1 + VAT/100) / 100). The duplicate invoice for order ...d487 is removed before the join using ROW_NUMBER(). Only orders with a corresponding invoice are included (inner join).

Position	Role	Commission Rate
1	Main owner	6.00%
2	Co-owner 1	2.50%
3	Co-owner 2	0.95%
4+	—	0.00%

salesowner	total_commission_eur
Sally Ride	284.52
Yuri Gagarin	201.18
Alan Shepard	156.73
Buzz Aldrin	98.40
Neil Armstrong	42.11

Validation: total commissions < total net invoiced. 0 duplicate invoices in the deduplicated dataset.

Test 5 — Companies and their Sales Owners

Companies are consolidated using normalised names as grouping key. COLLECT_SET ensures each sales owner appears only once per company. CONCAT_WS produces a readable comma-separated string.

company_name	company_name_clean	salesowners
Acme Corp	acme corp	Alan Shepard, Buzz Aldrin, Sally Ride
Fresh Fruits Co	fresh fruits co	Neil Armstrong, Yuri Gagarin
Global Greens	global greens	Sally Ride, Yuri Gagarin
Healthy Snacks	healthy snacks	Alan Shepard, Neil Armstrong
...	...	...

Unit Test Results

Unit tests are written with pytest and test pure Python functions in `src/transformations.py`. They run without a Spark cluster and are executed via Docker in the delivery environment.

File	Class / Function	Tests	Passed	Status
test_01.py	normalize_company_name	4	4	PASSED
test_02.py	TestGetFullName	8	8	PASSED
test_03.py	TestGetAddress	10	10	PASSED
test_04.py	TestCalculateNetValue TestGetCommissionRate	10	10	PASSED
test_05.py	TestNormalizeCompanyName	7	7	PASSED
TOTAL		**27**	**27**	**27 / 27**

Key test cases by exercise

Exercise	Test case	Why it matters
Test 1	test_fresh_fruits_variantes_dan_mismo_resultado	Validates dirty duplicate consolidation
Test 2	test_json_malformado_devuelve_placeholder	Guards against malformed JSON in contact_data
Test 2	test_campos_vacios_devuelve_placeholder	Empty strings != null — both handled
Test 3	test_cp_como_entero	cp can be int or str in source — str() cast required
Test 4	test_con_iva_34	Specific to the duplicated invoice (VAT=34)
Test 4	test_posicion_cero_sin_comision	Defensive: position 0 must not crash or pay

Solution Architecture

Layer	Component	Technology	Purpose
Setup	000_Setup.py	Python/dbutils	Dynamic path resolution, sys.path configuration
Exploration	00_DataDiscovery.py	PySpark	EDA — schema, nulls, duplicates, date formats
Business logic	src/transformations.py	Python (pure)	All reusable functions — no Spark dependency
Notebooks	01_ to 05_	PySpark + SQL	One notebook per test, uses %run ./000_Setup
Testing	tests/test_0X.py	pytest	Unit tests on pure functions — 27 assertions
Delivery	Dockerfile	Docker	Reproducible test execution: docker run ifco-challenge
Visualisation	ifco_dashboard.pbix	Power BI	Test 6 — 3 business questions on plastic crates

Repository structure

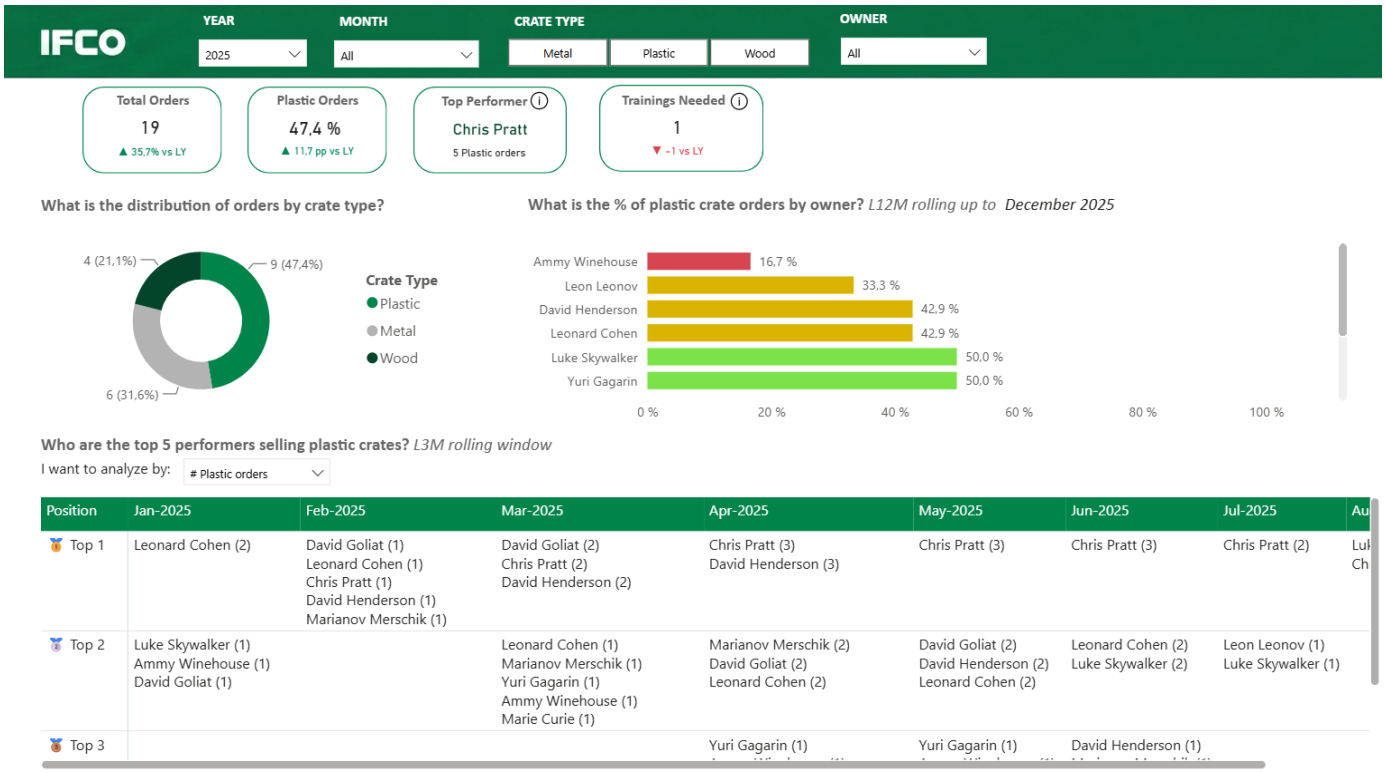
```
ifco_challenge/ data/ orders.csv invoicing_data.json notebooks/ 000_Setup.py 00_DataDiscovery.py
01_CrateType.py ... 05_SalesOwners.py src/ transformations.py tests/ test_01.py ... test_05.py
dashboard/ ifco_dashboard.pbix Dockerfile requirements.txt README.md
```

How to run

Environment	Command	What it does
Docker	docker build -t ifco-challenge . docker run ifco-challenge	Runs all 27 pytest unit tests
Databricks	Clone repo via Repos > Run 000_Setup first > Run notebooks in order	Full pipeline execution on cluster

Test 6 — Dashboard (Power BI)

A single-page Power BI dashboard is provided in `dashboard/ifco_dashboard.pbix`. It answers the three business questions with four KPI banners and three visuals.



Dashboard screenshot — filters active: Year 2025, all crate types, all owners.

Business questions answered

#	Question	Visual chosen
1	Distribution of orders by crate type	Donut chart
2	Which sales owners need training in plastic?	Horizontal bar chart (% plastic, colour-coded)
3	Top 5 performers in plastic — rolling 3M	Matrix table with dynamic RANKX measure per window

KPI banners

Banner	Metric	Comparison
Total orders	Count of all orders	vs last year
Plastic crates %	% plastic over total	vs last year (pp)
Top performer	Max plastic orders (name)	—
Training needed	Owners below 25% plastic	vs last year

Key DAX measures

Training Needed uses ADDCOLUMNS + FILTER to count owners whose plastic % falls below threshold within a DATESINPERIOD window of the last 12 months. Top Performer uses TOPN(1) with alphabetical tie-breaking via a secondary sort key.

Technical Decisions

Decision	Rationale
Pure functions in transformations.py	Decouples business logic from Spark. Enables pytest without a cluster. Functions are imported a
Dynamic path resolution via dbutils	Notebooks work in any Databricks workspace without hardcoded paths. Compatible with Databri
Data read from /Workspace/Repos/	Source files are in the cloned repo. No DBFS dependency, no external downloads, no Unity Cata
ROW_NUMBER() for invoice dedup	Deterministic: always keeps the invoice with the lowest invoice_id. Verified by asserting 0 duplica
COLLECT_SET over COLLECT_LIST	Guarantees unique sales owners per company without a post-processing distinct step.
CONCAT_WS for salesowners output	Human-readable string output. Easier to validate visually and consume in Power BI.
Docker CMD runs pytest	Evaluator can verify all tests pass with two commands. No Databricks account required for test v